



โครงการ การเขียนโปรแกรมเชิงอ็อบเจกต์ข้อมูล
โปรแกรมการจองรถ-ส่งจาก SUT-ไปตัวเมืองนครราชสีมา

จัดทำโดย

นายคนธ์พงษ์ เขาวนัธนาภิตต์ B6701598

นายวิธวินท์ ระวังจันทริต B6703165

นายชิตพัทธ์ สีสุด B6703271

นายวีรฉัตร จินะปรีวัตอาภรณ์ B6703370

นายวรรณเรศ ขุมพลกรัง B6728120

เสนอ

ผศ. ดร.ศุภกฤษฎี นวัตกรรมกุล

ผศ. ดร.จิตติมนต์ อังสกุล

อ. ดร.ธรรมศักดิ์ เขียรนิเวศน

รายงานนี้เป็นส่วนหนึ่งของรายวิชา
1101093 โครงการเขียนโปรแกรมเชิงอ็อบเจกต์และโครงสร้างข้อมูล
ภาคการศึกษาที่ 3 ปีการศึกษา 2563

สาขาวิชาเทคโนโลยีดิจิทัล กลุ่มหลักสูตรศาสตร์และศิลป์ดิจิทัล มหาวิทยาลัยเทคโนโลยีสุรนารี

รายละเอียดการแบ่งงานภายในกลุ่มที่ 18

ลำดับ	รหัสนักศึกษา	ชื่อ-นามสกุล	หัวข้อรับผิดชอบ	ร้อยละ
19	B6701598	นายคนธ์พงษ์ เขาวนธนาภิตติ์	1) ออกแบบ Class Diagram และโครงสร้าง Model ของระบบ 2) พัฒนา SystemManager สำหรับจัดการข้อมูลกลางของระบบ 3) พัฒนา ระบบบันทึก/โหลดข้อมูล (File I/O) ด้วยไฟล์ฐานข้อมูล	20%
92	B6703165	นายวิธวินท์ ระวังังห์หรีด	1) ออกแบบและพัฒนา GUI ฝั่งผู้โดยสาร 2) พัฒนา ระบบ Login / Register สำหรับผู้โดยสาร 3) แสดงตารางเดินรถ สถานะรถ และข้อมูลการจองฝั่งผู้ใช้	20%
96	B6703271	นายชิติพัทธ์ สีสุต	1) ออกแบบและพัฒนา GUI ฝั่งคนขับ 2) พัฒนา ระบบ Login / Register สำหรับคนขับ 3) เชื่อมการแสดงผลข้อมูลรอบเวลาและสถานะการเดินรถ	20%
97	B6703370	นายวีรฉัตร จินะปริวัตอารมณ์	1) พัฒนา ระบบจัดการคิวผู้โดยสารและฝั่งที่นั่ง 2) พัฒนา Logic การคำนวณเวลาเดินทางและสถานะรถ (Station Status) 3) พัฒนา ระบบ Booking การจองที่นั่งและการตัดสต็อกที่นั่ง	20%
148	B6728120	นายวรรณเรศ ขุมพลกรัง	1) ออกแบบ ธีมสี และ Custom Components เช่น RoundedPanel, ปุ่มโค้ง 2) พัฒนา Custom Dialog / Popup แจ้งเตือน 3) จัดทำ Graphic Assets (Icons) และจัดทำ รูปเล่มรายงานโครงการ	20%
รวมร้อยละทั้งหมด				100%

โปรแกรมการจองขนส่งจาก SUT-ไปตัวเมืองนครราชสีมา

ที่มาและความสำคัญ

การเดินทางระหว่างมหาวิทยาลัยเทคโนโลยีสุรนารี SUT และเขตตัวเมืองนครราชสีมาถือเป็นเส้นทางสำคัญที่มีนักศึกษาและบุคลากรใช้งานเป็นจำนวนมากในการเดินทางทำธุระหรือท่องเที่ยวและสำหรับนักศึกษาที่ไม่มียานพาหนะส่วนตัวในการเดินทางระบบการจัดการขนส่งในปัจจุบันอาจยังขาดความแน่นอนและประสิทธิภาพผู้โดยสารที่ต้องการจองล่วงหน้าเพื่อรับประกันที่นั่งอาจไม่มีระบบการรองรับที่ชัดเจน

โครงการ"โปรแกรมการจองขนส่งจาก SUT ไปตัวเมืองนครราชสีมา"จึงถูกพัฒนาขึ้นเพื่อตอบสนองต่อปัญหาดังกล่าว โดยมีที่มาจากการบูรณาการองค์ความรู้ทางทฤษฎีจากรายวิชา OBJECT-ORIENTED PROGRAMMING AND DATA STRUCTURES ซึ่งเป็นกรณีศึกษาความรู้ในการนำหลักการ การเขียนโปรแกรมเชิงวัตถุ (OOP) มาประยุกต์ใช้เพื่อจำลององค์ประกอบที่ซับซ้อนของระบบ เช่น การจอง (ซึ่งมีทั้งแบบจองล่วงหน้าและจองระหว่างทาง), ยานพาหนะ (ที่ต้องจัดการที่นั่งว่าง) และ เส้นทาง (ที่มีจุดจอดคงที่) รวมถึงการ เลือกใช้ โครงสร้างข้อมูล (Data Structures) ที่เหมาะสม เพื่อจัดการคิวผู้โดยสารและบริหารจัดการที่นั่งว่างได้ อย่างรวดเร็วและมีประสิทธิภาพ

ในเชิงประยุกต์ โครงการนี้มุ่งสร้างเครื่องมือที่ใช้งานได้จริงเพื่อยกระดับการเดินทางในเส้นทางนี้ โดยการพัฒนาการจองแบบผสม หรือ "Hybrid Booking" ที่ช่วยให้ผู้ใช้สามารถจองที่นั่งล่วงหน้าได้ และยังเพิ่มความยืดหยุ่นให้ผู้โดยสารตามจุดจอดสามารถใช้โปรแกรมตรวจสอบและจองที่นั่งว่างที่เหลืออยู่บนรถที่กำลังให้บริการได้ (On-Route Hailing) ซึ่งระบบนี้จะช่วยให้ผู้ขับขี่มีข้อมูลการรับ-ส่งผู้โดยสารที่ชัดเจนและผู้โดยสารได้รับความ สะดวกสบายและมีความแน่นอนในการเดินทางมากขึ้น

วัตถุประสงค์

1. เพื่อวิเคราะห์ ออกแบบ และพัฒนาระบบ "โปรแกรมการจองขนส่งจาก SUT ไปตัวเมืองนครราชสีมา" ที่สามารถจัดการการจองและติดตามสถานะได้
2. เพื่อประยุกต์ใช้หลักการออกแบบเชิงวัตถุ (OOP Principles) ในการสร้างแบบจำลององค์ประกอบต่างๆ ภายในระบบ เช่น ผู้ใช้, ผู้ขับขี, การจอง และยานพาหนะ
3. เพื่อศึกษาและเลือกใช้โครงสร้างข้อมูล (DataStructures) ที่เหมาะสมในการจัดเก็บและประมวลผล ข้อมูลการจอง, รอบเวลา และที่นั่งว่างอย่างมีประสิทธิภาพ
4. เพื่อพัฒนาระบบการจองแบบ "จองล่วงหน้า" (Pre-Booking) ที่รองรับการจองรอบเวลาและที่นั่งผ่านโปรแกรม
5. เพื่อสร้างส่วนต่อประสานกับผู้ใช้ (User Interface) ที่อำนวยความสะดวกให้ผู้ขับขีในการตรวจสอบ ผู้โดยสาร และให้ผู้ให้บริการในการจองรอบเวลา

ขอบเขตโครงการ

1. ระบบให้บริการจองการเดินทางในเส้นทางระหว่างมหาวิทยาลัยเทคโนโลยีสุรนารี (SUT) และจุดจอดที่กำหนดในเขตตัวเมืองนครราชสีมาเทคโนโลยีธานี, เดอะมอลล์, เทอมินอล21, บขส.ใหม่, เซ็นทรัลลานท้าวสุรนารี) โดยรองรับทั้งขาไปและขากลับ
2. ระบบใช้โมเดล "การจองล่วงหน้า (Pre-Booking)" เท่านั้น
 - การจองล่วงหน้า (Pre-Booking): ผู้ใช้จองรอบเวลาและที่นั่งล่วงหน้าผ่านโปรแกรม
3. ผู้ใช้บริการต้องทำการจองรอบเวลาและที่นั่งล่วงหน้าผ่านโปรแกรมเท่านั้นโดยสามารถตรวจสอบจำนวนที่นั่งว่างที่เหลืออยู่ในแต่ละรอบเวลาได้
4. ระบบไม่อนุญาตให้มีการโบกรถจากข้างทาง (Street-hailing) หรือการจองระหว่างทาง (On-Route Hailing)
การเข้าสู่ระบบ (Authentication) (หน้าแรกของโปรแกรม)
 - การเข้าสู่ระบบ (Login)
 - การลงทะเบียน (Register)
 - (หลังจาก Login สำเร็จ ระบบจะตรวจสอบว่าเป็น "คนขับ" หรือ "ผู้ใช้" แล้วแสดงเมนูหลัก)

เมนูหลักของระบบของคนขับมีดังนี้

1. เมนูจัดการรอบเวลา (Schedule Management)
 - คนขับต้องใช้เมนูนี้เพื่อ "เปิด" ที่ตนจะเริ่มให้บริการ
2. เมนูเช็คชื่อผู้โดยสาร (Check Passengers)
 - แสดงรายชื่อผู้โดยสาร จุดขึ้น-ลง สีเขียว/สีแดง สำหรับ "รอบปัจจุบัน"
 - เมนูประวัติและสถิติ (History & Statistics) แสดงรายการรอบเวลาที่เคยขับไปแล้ว และสถิติการรับงาน
3. ออกจากระบบ (Logout)

ความคิดสร้างสรรค์ในการออกแบบ (Creativity)

ระบบ SUT-TRANSPORT มีความคิดสร้างสรรค์ในการออกแบบโดยนำ สีสันประจำมหาวิทยาลัยเทคโนโลยีสุรนารี มาใช้เป็นแนวคิดหลักในการออกแบบส่วนติดต่อผู้ใช้ เพื่อสร้างเอกลักษณ์และความเชื่อมโยงกับมหาวิทยาลัย นอกจากนี้ยังมีการออกแบบหน้าจอการใช้งานให้เรียบง่าย เข้าใจง่าย และแยกบทบาทการใช้งานระหว่างผู้ใช้และคนขับอย่างชัดเจน รวมถึงการใช้สีและสัญลักษณ์เพื่อแสดงสถานะต่าง ๆ ของระบบ ช่วยให้ผู้ใช้สามารถเรียนรู้และใช้งานระบบได้สะดวก

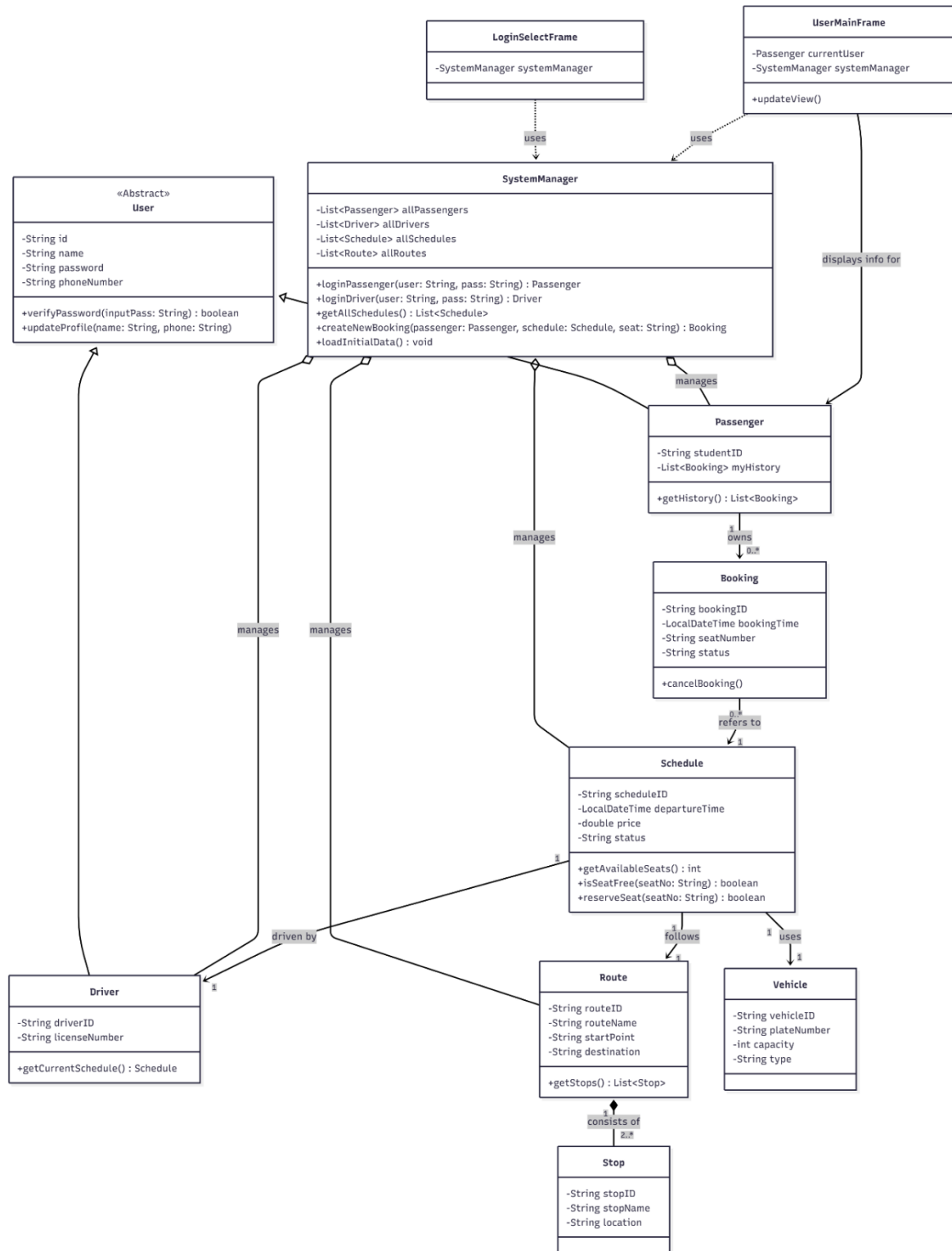
ผลกระทบต่อชุมชน (Impact)

SUT-TRANSPORT ช่วยอำนวยความสะดวกในการเดินทางระหว่างมหาวิทยาลัยและตัวเมืองนครราชสีมา ส่งผลให้การเดินทางมีความเป็นระเบียบมากขึ้น ลดความแออัดและความสับสนในการใช้บริการรถรับ-ส่ง ช่วยประหยัดเวลาและลดภาระค่าใช้จ่ายของนักศึกษาและบุคลากร อีกทั้งยังส่งเสริมการใช้ระบบขนส่งร่วมกัน ซึ่งช่วยลดปริมาณการใช้รถส่วนตัวและเป็นผลดีต่อชุมชนโดยรวม

ประโยชน์ที่คาดว่าจะได้รับ

ช่วยเพิ่มประสิทธิภาพในการบริหารจัดการรถรับ-ส่งและข้อมูลการจองลดความผิดพลาดในการจัดการผู้โดยสารผู้โดยสารสามารถตรวจสอบและจองการเดินทางได้ล่วงหน้าอย่างสะดวกขณะที่คนขับสามารถจัดการเส้นทางและผู้โดยสารได้อย่างเป็นระบบ นอกจากนี้ข้อมูลสถิติที่ได้จากระบบยังสามารถนำไปใช้ในการวิเคราะห์และพัฒนาระบบให้บริการขนส่งในอนาคตได้

Class Diagram



คู่มือการใช้งานโปรแกรมการจองขนส่งจาก SUT ไปตัวเมืองนครราชสีมา

ถึงดาวน์โหลดโปรแกรม Eclipse IDE: [คลิกตรงนี้](#)

ถึงดาวน์โหลดโปรแกรม Sut-Transport : [คลิกตรงนี้](#)

ถึงวิดีโอสอนติดตั้งโปรแกรมทั้ง2สองโปรแกรม : [คลิกตรงนี้](#)

ส่วนของผู้ใช้

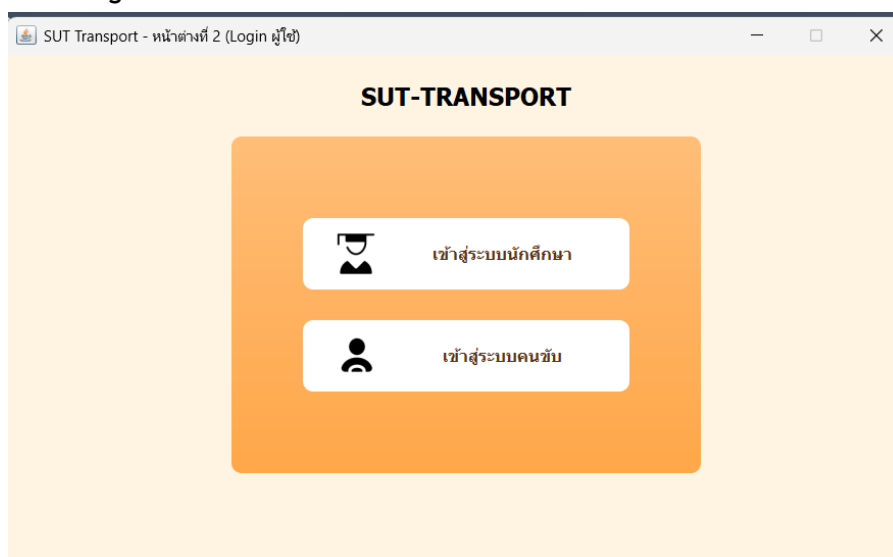
1. หน้าจอสมัครสมาชิก(Register)

หน้าจอนี้ใช้สำหรับผู้ใช้งานที่ยังไม่มีบัญชีในระบบ SUT-TRANSPORT เพื่อทำการสมัครสมาชิกก่อนเข้าสู่ระบบ โดยผู้ใช้งานจะต้องกรอกข้อมูลให้ครบถ้วนตามช่องที่กำหนด ดังนี้

- ชื่อผู้ใช้ ใช้สำหรับระบุชื่อบัญชีผู้ใช้งาน
- รหัสนักศึกษา ใช้สำหรับระบุรหัสประจำตัวนักศึกษา
- Email ใช้สำหรับติดต่อและยืนยันข้อมูลผู้ใช้งาน
- รหัสผ่าน ใช้สำหรับตั้งรหัสผ่านเพื่อเข้าสู่ระบบ

เมื่อกรอกข้อมูลครบถ้วนแล้ว ให้กดปุ่ม “สมัครสมาชิก” เพื่อบันทึกข้อมูลเข้าสู่ระบบ

2. หน้าจอล็อกอิน (Login)



หน้าจอนี้เป็นหน้าจอเริ่มต้นของโปรแกรม SUT-TRANSPORT ซึ่งใช้สำหรับเลือกประเภทผู้ใช้งานก่อนเข้าสู่ระบบ โดยระบบจะแบ่งผู้ใช้งานออกเป็น 2 ประเภทหลัก ได้แก่

1. เข้าสู่ระบบนักศึกษา
2. เข้าสู่ระบบคนขับ

ผู้ใช้งานสามารถเลือกประเภทที่ตรงกับตนเองได้โดยคลิกที่ปุ่มที่ต้องการ

3. หน้าจอเข้าสู่ระบบผู้ใช้งาน (User Login)

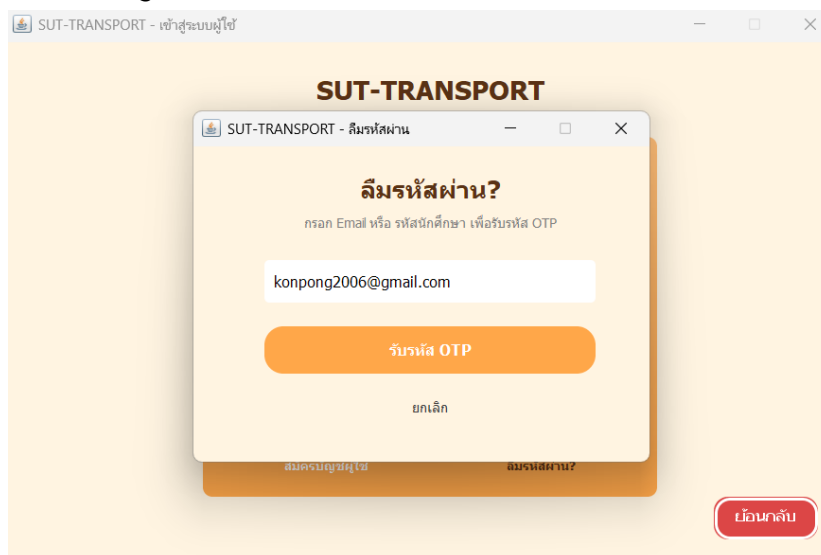
The image displays two screenshots of the SUT-TRANSPORT user login interface. The top screenshot shows the initial login screen with the title 'SUT-TRANSPORT' and a 'เข้าสู่ระบบ' (Login) button. Below the button are two input fields: 'รหัสนักศึกษา :' (Student ID) and 'รหัสผ่าน :' (Password). A 'สมัครบัญชีผู้ใช้' (Sign up) link is visible at the bottom. The bottom screenshot shows the same interface but with an error message 'Email หรือ รหัสผ่าน ไม่ถูกต้อง' (Email or Password is incorrect) displayed in a red box. A 'ตกลง' (OK) button is present next to the error message. The 'เข้าสู่ระบบ' (Login) button is now greyed out. A 'ลืมรหัสผ่าน?' (Forgot password?) link is also visible at the bottom right.

หน้าจอนี้เป็นหน้าจอสำหรับเข้าสู่ระบบของผู้ใช้งานในระบบ SUT-TRANSPORT โดยจะแสดงแบบฟอร์มสำหรับกรอกข้อมูลเพื่อยืนยันตัวตนก่อนใช้งานระบบถ้าใส่ข้อมูลยืนยันผิดพลาดระบบจะแสดงว่า Email หรือ รหัสผ่านไม่ถูกต้อง ผู้ใช้งานจะต้องกรอกข้อมูลให้ครบถ้วน ดังนี้

- รหัสนักศึกษา
- รหัสผ่าน

หลังจากกรอกข้อมูลเรียบร้อยแล้ว ให้กดปุ่ม “เข้าสู่ระบบ” เพื่อทำการเข้าสู่ระบบ

4. หน้าจอลืมรหัสผ่าน (Forgot Password)

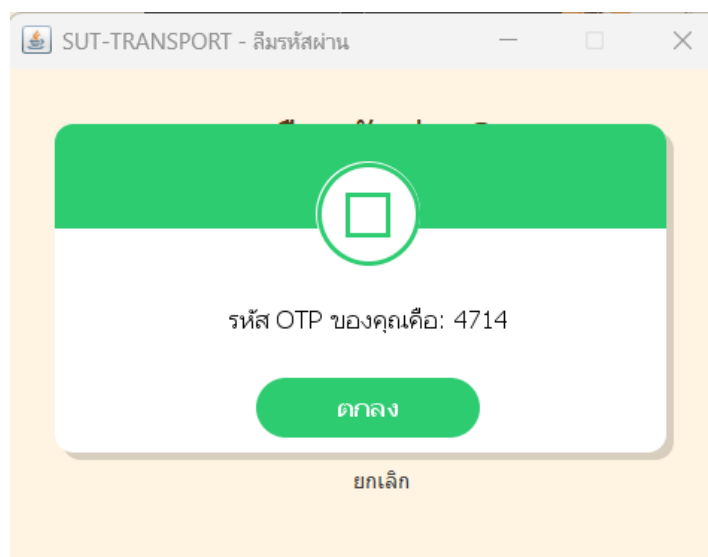


หน้าจอนี้ใช้สำหรับผู้ใช้งานที่ลืมรหัสผ่าน โดยผู้ใช้ต้องกรอก Email หรือรหัสนักศึกษา ที่เคยลงทะเบียนไว้ในระบบ จากนั้นกดปุ่ม รับรหัส OTP เพื่อเริ่มกระบวนการยืนยันตัวตนระบบ otp นี้ เป็น ระบบ simulation ขึ้น มาโดยใช้การ สุ่ม รหัส ตัว เลข 4 หลักขึ้น มา ระบบลืมรหัสผ่านใช้สำหรับผู้ใช้ที่ไม่สามารถเข้าสู่ระบบได้ โดยมีขั้นตอนดังนี้

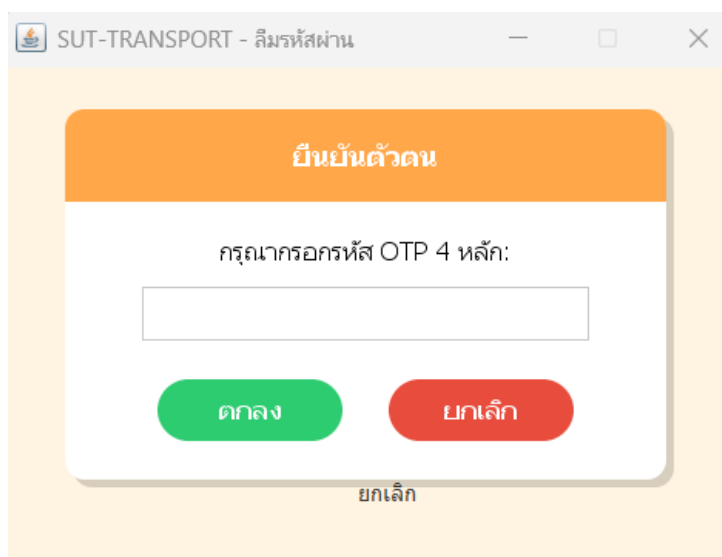
1. รับรหัส OTP
2. ยืนยันรหัส OTP
3. ตั้งรหัสผ่านใหม่

เมื่อดำเนินการสำเร็จ ผู้ใช้สามารถกลับไปเข้าสู่ระบบด้วยรหัสผ่านใหม่ได้ทันที

4.1 รับรหัส OTP

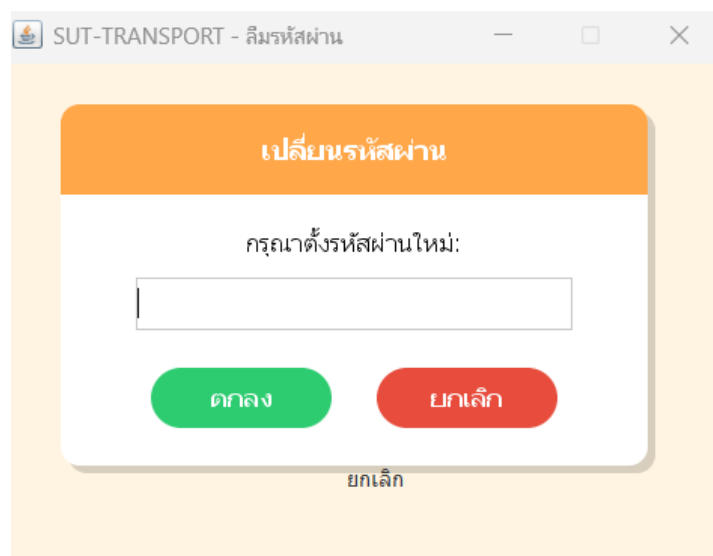


4.2 ยืนยันรหัส OTP



The screenshot shows a web browser window titled "SUT-TRANSPORT - ลืมรหัสผ่าน". The main content area has an orange header with the text "ยืนยันตัวตน" (Verify Identity). Below the header, the text "กรุณามกรหัส OTP 4 หลัก:" (Please enter the 4-digit OTP code:) is displayed. There is a text input field for the OTP code. Below the input field are two buttons: a green button labeled "ตกลง" (OK) and a red button labeled "ยกเลิก" (Cancel). At the bottom of the form, there is a link labeled "ยกเลิก" (Cancel).

4.3 ตั้งรหัสผ่านใหม่

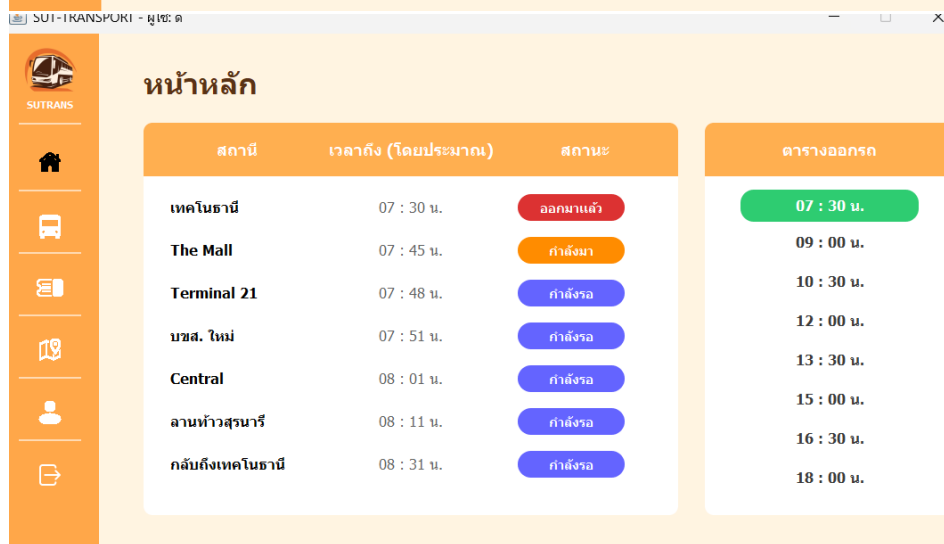
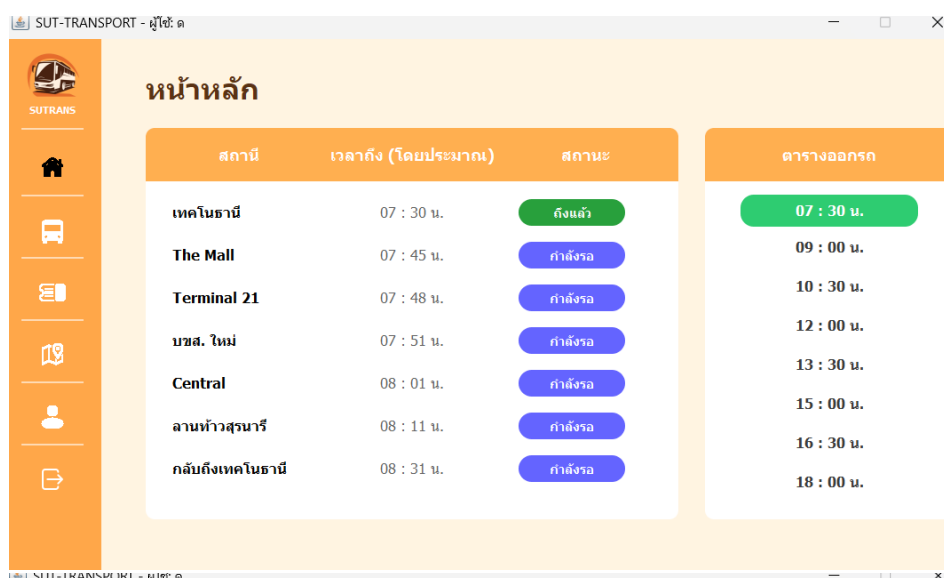


The screenshot shows a web browser window titled "SUT-TRANSPORT - ลืมรหัสผ่าน". The main content area has an orange header with the text "เปลี่ยนรหัสผ่าน" (Change Password). Below the header, the text "กรุณาตั้งรหัสผ่านใหม่:" (Please set a new password:) is displayed. There is a text input field for the new password. Below the input field are two buttons: a green button labeled "ตกลง" (OK) and a red button labeled "ยกเลิก" (Cancel). At the bottom of the form, there is a link labeled "ยกเลิก" (Cancel).

5. หน้าจอหลักผู้ใช้งาน (User Home)



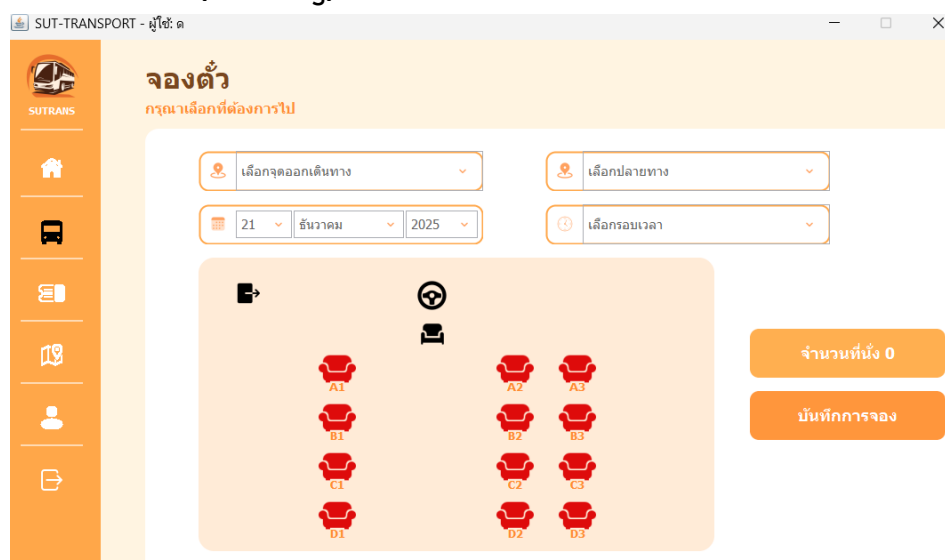
ในกรณีของผู้ใช้ ถ้าคนขับยังไม่กดเลือกรอบเวลา แล้วก็ยังไม่เปลี่ยนแบบนี้



หน้าจอนี้เป็นหน้าหลักของผู้ใช้งานหลังจากเข้าสู่ระบบสำเร็จใช้สำหรับแสดงข้อมูลการเดินทางจากมหาวิทยาลัยเทคโนโลยีสุรนารีไปยังตัวเมืองนครราชสีมา โดยจะแสดงรายชื่อสถานที่ปลายทาง สถานะของรถและตารางเวลาออกรถเพื่อให้ผู้ใช้งานสามารถตรวจสอบและวางแผนการเดินทางได้อย่างสะดวกซึ่งมีการบอกสถานะทั้งหมด 4 สถานะได้แก่

1. สีเทา : ยังไม่มีสถานะ
2. สีเขียว : ถึงสถานีนั่นแล้ว
3. สีเหลือง : กำลังไปสถานีนั่นแล้ว
4. สีแดง : ออกมาจากสถานีนั่นแล้ว

6. หน้าจอจองการเดินทาง (Booking)



ใช้สำหรับจองการเดินทาง โดยผู้ใช้งานเลือกต้นทาง ปลายทาง วันที่ เวลา และเลือกที่นั่งที่ต้องการ จากนั้นกดบันทึกการจอง ระบบจะทำการบันทึกข้อมูลการจองให้เรียบร้อย

7. หน้าจอประวัติการจอง (Booking History)

SUT-TRANSPORT - ผู้ใช้: TK

ประวัติการจอง

วันที่ (ใหม่ -> เก่า).. ค้นหา...

วันที่จอง	เวลาที่จอง	จุดเริ่ม	ปลายทาง	ที่นั่ง	ลำดับ	สถานะ	จัดการ
03 มกราคม 2026	15 : 00 น.	บขส. ใหม่	กลับถึงเขตในธานี	10	A2, B2, B3, A3...	สำเร็จ	ยืนยันแล้ว
29 ธันวาคม 2025	15 : 00 น.	เขตในธานี	Central	6	A2, A3, C3, C2...	รอดำเนินการ	ยกเลิก
27 ธันวาคม 2025	16 : 30 น.	The Mall	Central	6	A2, A3, A1, B3...	รอดำเนินการ	ยกเลิก
27 ธันวาคม 2025	09 : 00 น.	เขตในธานี	บขส. ใหม่	8	A2, A3, B2, B3...	รอดำเนินการ	ยกเลิก
26 ธันวาคม 2025	16 : 30 น.	ลานท้าวสุรนารี	กลับถึงเขตในธานี	4	C1, D1, C3, C2	รอดำเนินการ	ยกเลิก
26 ธันวาคม 2025	13 : 30 น.	เขตในธานี	ลานท้าวสุรนารี	4	C1, A1, A2, A3	สำเร็จ	ยืนยันแล้ว
25 ธันวาคม 2025	18 : 00 น.	Central	กลับถึงเขตในธานี	8	A1, A2, A3, C1...	รอดำเนินการ	ยกเลิก
25 ธันวาคม 2025	15 : 00 น.	เขตในธานี	Central	5	A1, A2, C2, C3...	รอดำเนินการ	ยกเลิก
23 ธันวาคม 2025	18 : 00 น.	Terminal 21	กลับถึงเขตในธานี	4	A2, A3, C3, C2	รอดำเนินการ	ยกเลิก

SUT-TRANSPORT - ผู้ใช้: TK

ประวัติการจอง

วันที่ (เก่า -> ใหม่).. ค้นหา...

วันที่ (ใหม่ -> เก่า)

วันที่ (เก่า -> ใหม่)

สถานะ

วันที่จอง	เวลาที่จอง	จุดเริ่ม	ปลายทาง	ที่นั่ง	ลำดับ	สถานะ	จัดการ
22 ธันวาคม 2025	09 : 00 น.	เขตในธานี	The Mall			รอดำเนินการ	ยกเลิก
22 ธันวาคม 2025	16 : 30 น.	The Mall	กลับถึงเขตในธานี	3	A2, A3, B3	รอดำเนินการ	ยกเลิก
23 ธันวาคม 2025	10 : 30 น.	เขตในธานี	Terminal 21	4	A1, C1, D2, D3	รอดำเนินการ	ยกเลิก
23 ธันวาคม 2025	18 : 00 น.	Terminal 21	กลับถึงเขตในธานี	4	A2, A3, C3, C2	รอดำเนินการ	ยกเลิก
25 ธันวาคม 2025	15 : 00 น.	เขตในธานี	Central	5	A1, A2, C2, C3...	รอดำเนินการ	ยกเลิก
25 ธันวาคม 2025	18 : 00 น.	Central	กลับถึงเขตในธานี	8	A1, A2, A3, C1...	รอดำเนินการ	ยกเลิก
26 ธันวาคม 2025	13 : 30 น.	เขตในธานี	ลานท้าวสุรนารี	4	C1, A1, A2, A3	สำเร็จ	ยืนยันแล้ว
26 ธันวาคม 2025	16 : 30 น.	ลานท้าวสุรนารี	กลับถึงเขตในธานี	4	C1, D1, C3, C2	รอดำเนินการ	ยกเลิก
27 ธันวาคม 2025	09 : 00 น.	เขตในธานี	บขส. ใหม่	8	A2, A3, B2, B3...	รอดำเนินการ	ยกเลิก

SUT-TRANSPORT - ผู้ใช้: TK

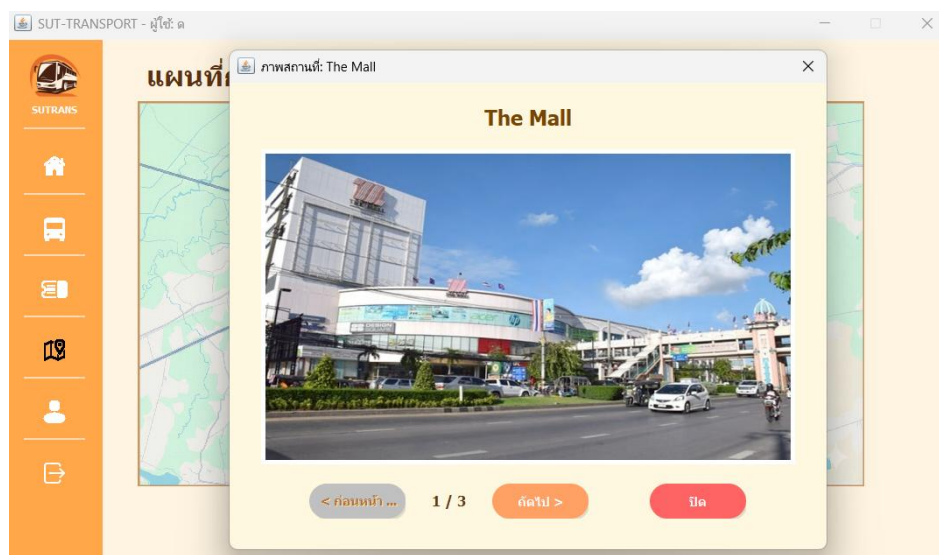
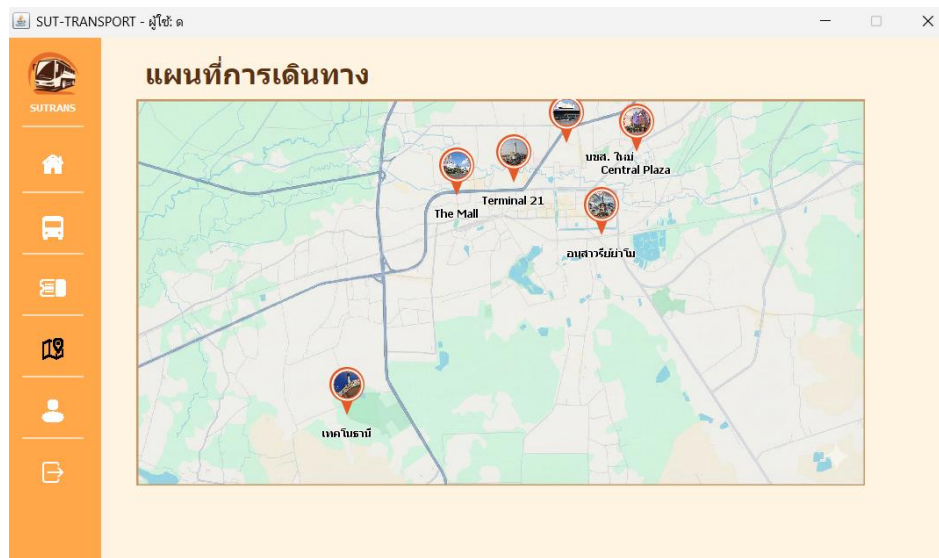
ประวัติการจอง

วันที่ (เก่า -> ใหม่).. 22 ค้นหา...

วันที่จอง	เวลาที่จอง	จุดเริ่ม	ปลายทาง	ที่นั่ง	ลำดับ	สถานะ	จัดการ
22 ธันวาคม 2025	09 : 00 น.	เขตในธานี	The Mall	3	A2, A3, B2	รอดำเนินการ	ยกเลิก
22 ธันวาคม 2025	16 : 30 น.	The Mall	กลับถึงเขตในธานี	3	A2, A3, B3	รอดำเนินการ	ยกเลิก

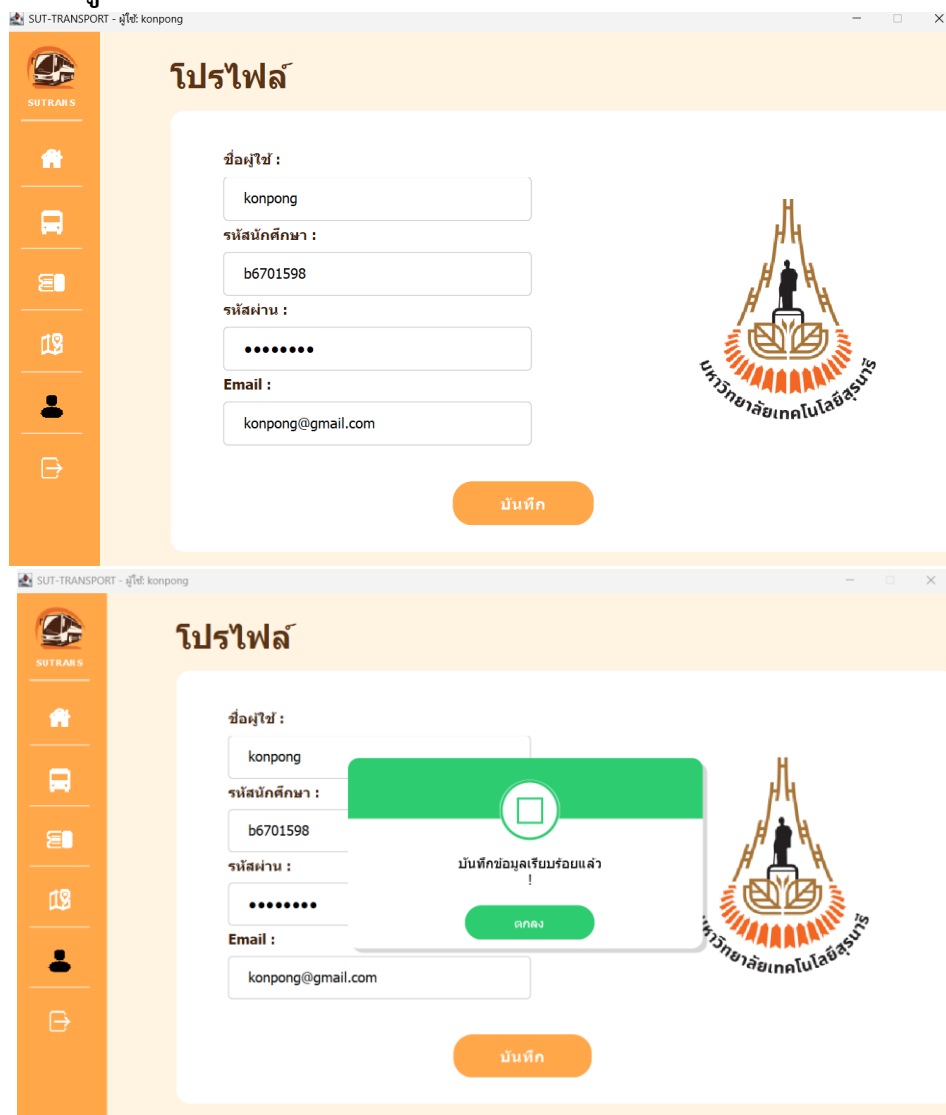
ใช้สำหรับแสดงประวัติการจองการเดินทางทั้งหมดของผู้ใช้งาน โดยแสดงข้อมูลในรูปแบบตาราง เช่น วันที่ เวลา จุดเริ่มต้น ปลายทาง จำนวนที่นั่ง และสถานะการจอง ผู้ใช้งานสามารถจัดเรียง ค้นหา และยกเลิกการจองได้ในกรณีที่ยังไม่ถึงรอบเดินทาง

8. หน้าจอแผนที่การเดินทาง (Travel Map)



หน้าจอนี้ใช้สำหรับแสดงเส้นทางการเดินรถจากมหาวิทยาลัยเทคโนโลยีสุรนารีไปยังจุดหมายต่าง ๆ ในตัวเมืองนครราชสีมา ผู้ใช้สามารถดูตำแหน่งสถานีทั้งหมดบนแผนที่ได้ เมื่อเลือกสถานี ระบบจะแสดงรายละเอียดของสถานีนั้น เช่น รูปภาพและลำดับจุดจอด เพื่อช่วยให้ผู้ใช้เข้าใจเส้นทางการเดินรถได้ง่ายและชัดเจน

9. หน้าจอโปรไฟล์ผู้ใช้งาน (User Profile)



SUT-TRANSPORT - ผู้ใช้: konpong

โปรไฟล์

ชื่อผู้ใช้ :
konpong

รหัสนักศึกษา :
b6701598

รหัสผ่าน :
.....

Email :
konpong@gmail.com

บันทึก

SUT-TRANSPORT - ผู้ใช้: konpong

โปรไฟล์

ชื่อผู้ใช้ :
konpong

รหัสนักศึกษา :
b6701598

รหัสผ่าน :
.....

Email :
konpong@gmail.com

บันทึกข้อมูลเรียบร้อยแล้ว !

ตกลง

บันทึก

หน้าจอนี้ใช้สำหรับแสดงและแก้ไขข้อมูลส่วนตัวของผู้ใช้งาน เช่น ชื่อผู้ใช้ รหัสนักศึกษา รหัสผ่าน และอีเมล ผู้ใช้งานสามารถแก้ไขข้อมูลและกดปุ่มบันทึกเพื่ออัปเดตข้อมูลในระบบได้

10. หน้าต่างยืนยันการออกจากระบบ (Logout Confirmation)

โปรไฟล์

ชื่อผู้ใช้ :
 ค
 รหัสนักศึกษา :
 รหัสผ่าน :
 •
 Email :
 บันทึก

ยืนยันที่จะออกจากระบบหรือไม่?

กลับ ยืนยัน

บันทึก

มหาวิทยาลัยเทคโนโลยีสุรนารี

หน้าต่างนี้ใช้สำหรับยืนยันการออกจากระบบ เมื่อผู้ใช้งานกดออกจากระบบ ระบบจะแสดงหน้าต่างแจ้งเตือนเพื่อให้ผู้ใช้งานเลือกยืนยันหรือยกเลิกการออกจากระบบ เพื่อป้องกันการออกจากระบบโดยไม่ตั้งใจ

ส่วนของคนขับ

11. หน้าจอเข้าสู่ระบบคนขับ (Driver Login)

SUT-TRANSPORT - เข้าสู่ระบบคนขับ

SUT-TRANSPORT

เข้าสู่ระบบ (คนขับ)

Email / Username :

รหัสผ่าน :

เข้าสู่ระบบ

สมัครบัญชีผู้ใช้

ย้อนกลับ

หน้าจอนี้ใช้สำหรับให้คนขับเข้าสู่ระบบ โดยกรอก Email หรือ Username และ รหัสผ่าน จากนั้นกดปุ่ม เข้าสู่ระบบ เพื่อเข้าใช้งานระบบในส่วนของคนขับ หากยังไม่มีบัญชีสามารถสมัครสมาชิกได้ หรือกดปุ่มย้อนกลับเพื่อกลับไปหน้าก่อนหน้า

12. หน้าจอสมัครสมาชิกคนขับ (Driver Register)



The screenshot shows a web browser window titled "SUT-TRANSPORT - สมัครสมาชิกคนขับ". The main heading is "SUT-TRANSPORT". Below it, the form is titled "สมัครสมาชิก (คนขับ)". The form contains four input fields: "ชื่อผู้ใช้ :", "Email :", "รหัสผ่าน :", and "หมายเลขรถ (Bus ID) :". At the bottom of the form is a button labeled "ลงทะเบียน". To the right of the form is a red button labeled "ย้อนกลับ".

หน้าจอนี้ใช้สำหรับสมัครสมาชิกในส่วนของคนขับ โดยกรอกข้อมูล ชื่อผู้ใช้, Email, รหัสผ่าน และหมายเลขรถ (Bus ID) จากนั้นกดปุ่ม ลงทะเบียน เพื่อบันทึกข้อมูลเข้าสู่ระบบ เมื่อสมัครสำเร็จสามารถนำข้อมูลไปใช้เข้าสู่ระบบคนขับได้ทันที

13. หน้าจอแจ้งจบการเดินทาง (End Trip Notification)

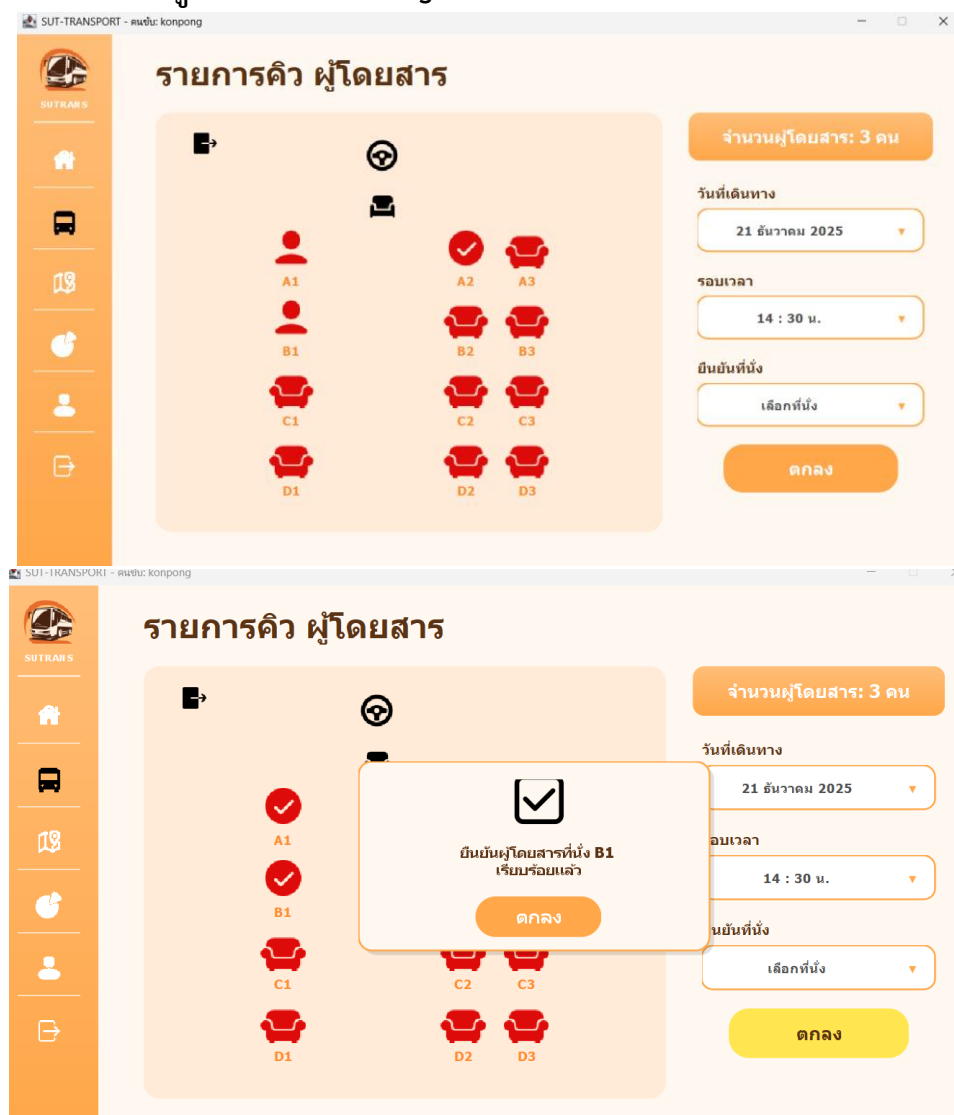


หน้าจอนี้ใช้สำหรับแสดงรายชื่อสถานีต่าง ๆ พร้อมจำนวนผู้โดยสารที่ขึ้นและลงในแต่ละสถานี เพื่อให้คนขับสามารถตรวจสอบและจัดการผู้โดยสารได้อย่างถูกต้อง

เมื่อคนขับกดปุ่ม “สถานีถัดไป” ระบบจะเปลี่ยนสถานีปัจจุบันเป็น สีส้ม เพื่อแสดงสถานีที่กำลังให้บริการอยู่ และจะเลื่อนไปยังสถานีถัดไปตามลำดับเส้นทาง

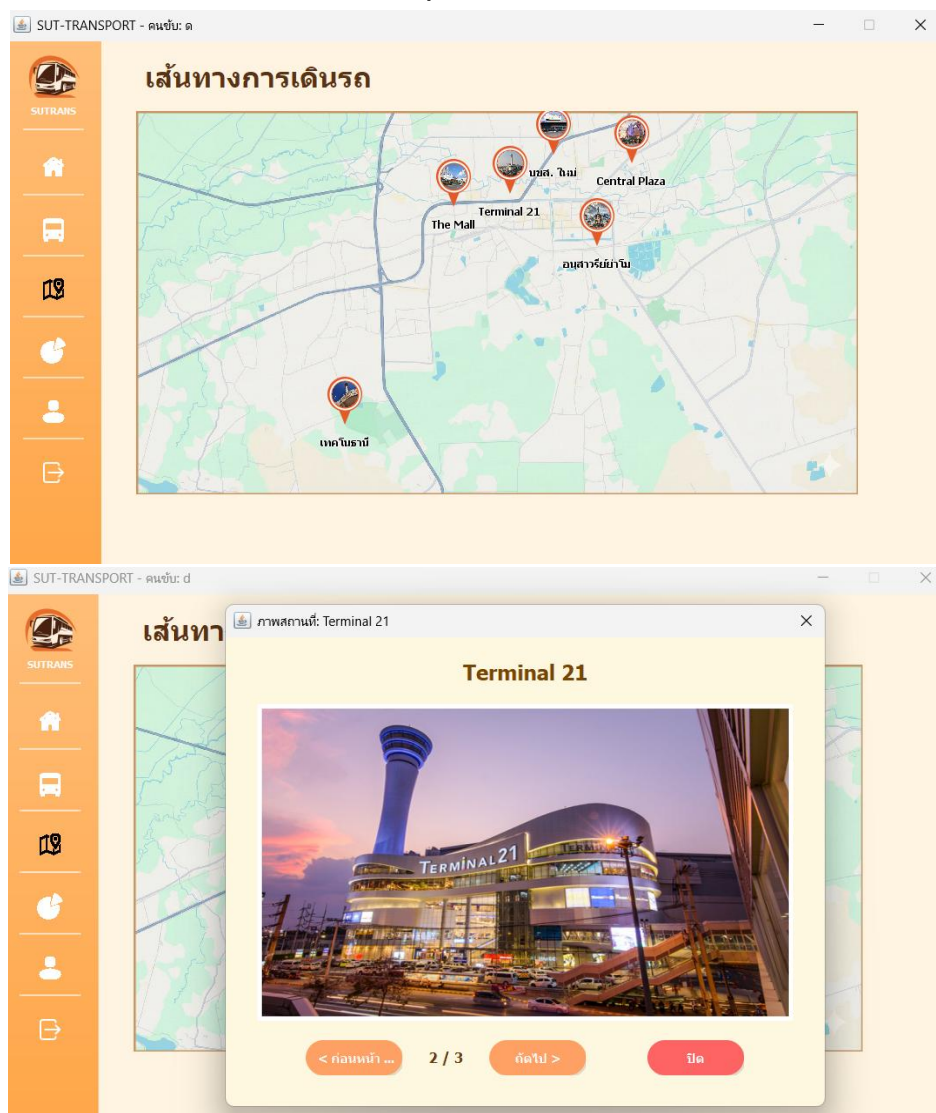
เมื่อกดปุ่ม “สถานีถัดไป” ต่อเนื่องไปเรื่อย ๆ จนถึงสถานีสุดท้าย ระบบจะแจ้งว่าการเดินทางในรอบนั้นสิ้นสุดลง และเมื่อกดยืนยัน ระบบจะเริ่มต้น รอบการเดินทางถัดไป โดยอัตโนมัติ

14. หน้าจอรายการคิวผู้โดยสาร (Passenger Queue)



หน้าจอนี้ใช้สำหรับแสดงรายการผู้โดยสารในรอบการเดินรถโดยจะแสดงจำนวนผู้โดยสารวันที่เดินทาง รอบเวลา และสถานะที่นั่ง เพื่อให้คนขับตรวจสอบและยืนยันผู้โดยสารที่ขึ้นรถในแต่ละรอบได้อย่างถูกต้อง เมื่อยืนยันเรียบร้อยแล้วสามารถกดปุ่มตกลงเพื่อดำเนินการต่อไปได้

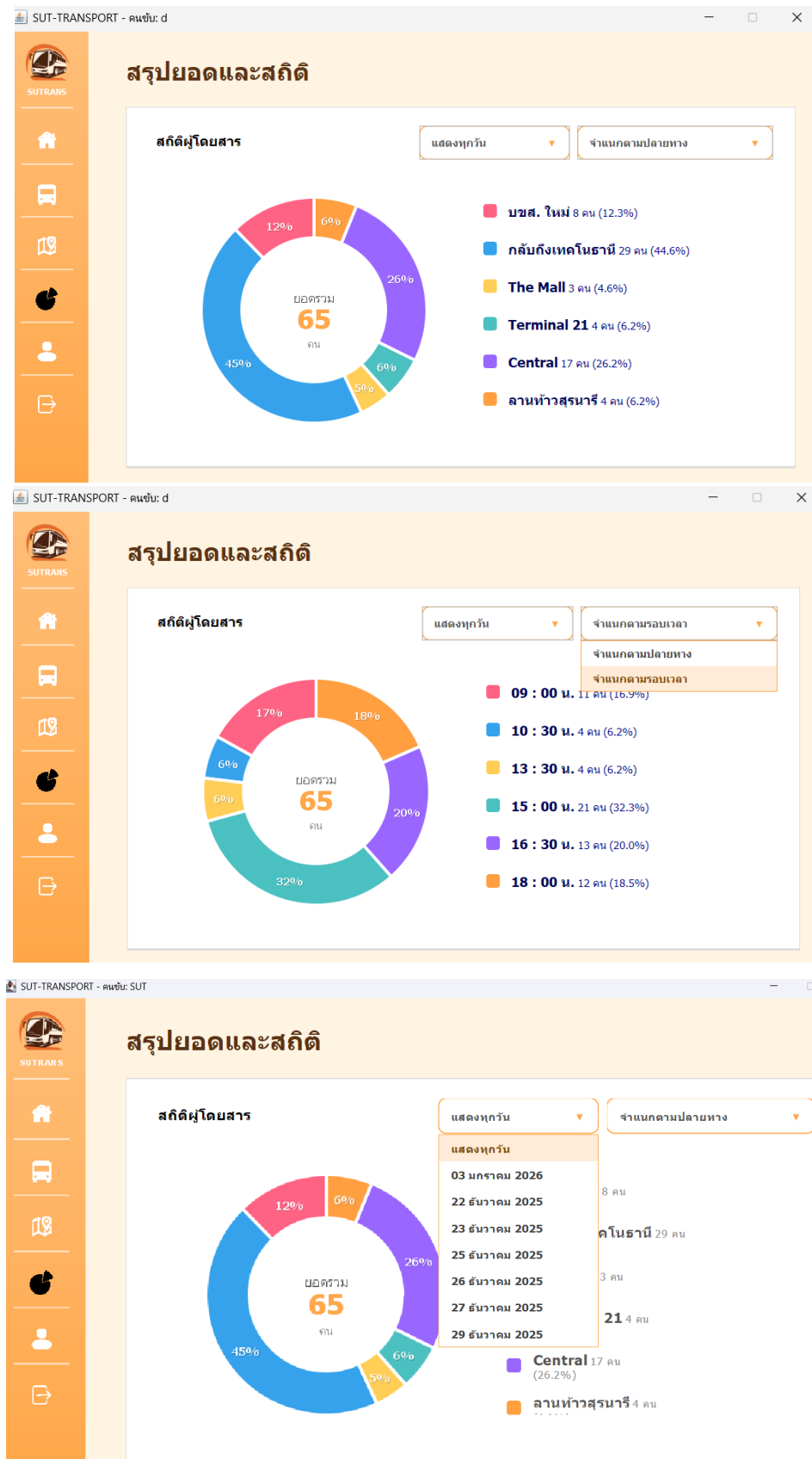
15. หน้าจอเส้นทางการเดินรถ (Route Map)



หน้าจอนี้ใช้สำหรับแสดงเส้นทางการเดินรถจากมหาวิทยาลัยเทคโนโลยีสุรนารีไปยังจุดหมายต่าง ๆ ในตัวเมืองนครราชสีมา ผู้ใช้สามารถดูตำแหน่งสถานีทั้งหมดบนแผนที่ได้

เมื่อเลือกสถานี ระบบจะแสดงรายละเอียดของสถานีนั้น เช่น รูปภาพและลำดับจุดจอด เพื่อช่วยให้ผู้ใช้เข้าใจเส้นทางการเดินรถได้ง่ายและชัดเจน

16. หน้าจอสรุปยอดและสถิติ (Summary & Statistics)



หน้าจอนี้ใช้สำหรับแสดงสรุปจำนวนผู้โดยสารและสถิติการเดินทางในแต่ละวันโดยแสดงข้อมูลในรูปแบบกราฟและเลือกดูสถิติในแต่ละที่มีการจองในแต่วัน, ประเภทต้นทาง-ปลายทาง เพื่อให้คนขับสามารถดูภาพรวมของจำนวนผู้โดยสารและปลายทางที่มีการใช้งานมากที่สุดได้อย่างชัดเจน มีตัวกรองข้อมูลที่สามารถดูเฉพาะวันนั้นได้สามารถเปลี่ยนจากสถานีปลายทางเป็นเวลาแทนได้

17. หน้าจอโปรไฟล์คนขับ (Driver Profile)

The image displays two screenshots of the 'โปรไฟล์' (Profile) page in the SUT-TRANSPORT system, accessed by user 'konpong'.

Top Screenshot (Initial Form):

- Username:** konpong
- รหัสนักศึกษา (Student ID):** b6701598
- รหัสผ่าน (Password):** [masked with dots]
- Email:** konpong@gmail.com
- Buttons:** บันทึกลง (Save), มหาวิทยาลัยเทคโนโลยีสุรนารี (Sukranart University Logo)

Bottom Screenshot (After Save):

- Username:** konpong
- รหัสนักศึกษา (Student ID):** 12345
- รหัสผ่าน (Password):** [masked with dots]
- Email:** konpong@gmail.com
- Message:** บันทึกข้อมูลเรียบร้อยแล้ว (Information saved successfully)
- Buttons:** บันทึก (Save), ตกลง (OK), มหาวิทยาลัยเทคโนโลยีสุรนารี (Sukranart University Logo)

หน้าจอนี้ใช้สำหรับแสดงและแก้ไขข้อมูลส่วนตัวของคนขับ เช่น ชื่อผู้ใช้ รหัสรถ รหัสผ่าน และอีเมล เมื่อแก้ไขข้อมูลแล้วสามารถกดปุ่มบันทึกเพื่ออัปเดตข้อมูลในระบบได้

18. หน้าจอยืนยันการออกจากระบบ (Logout Confirmation – Driver)

SUT-TRANSPORT - คนขับ: konpong

โปรไฟล์

ชื่อผู้ใช้ : konpong

รหัสรถบัส : 12345

รหัสผ่าน :

Email : konpong@gmail.com

ยืนยันที่จะออกจากระบบหรือไม่?

กลับ ยืนยัน

บันทึก

มหาวิทยาลัยเทคโนโลยีสุรนารี

หน้าตาจนี้ใช้สำหรับยืนยันการออกจากระบบของคนขับ เมื่อกดออกจากระบบ ระบบจะแสดงข้อความให้เลือก ยืนยัน เพื่่อออกจากระบบ หรือ กลับ เพื่อยกเลิกและใช้งานต่อ ป้องกันการออกจากระบบโดยไม่ตั้งใจ

การใช้แนวคิด Inheritance ในการพัฒนา

1. คลาส User (Superclass)

ระบบได้กำหนดให้คลาส User ซึ่งอยู่ในไฟล์ src/model/User.java เป็นคลาสแม่ (Superclass) สำหรับผู้ใช้งานทุกประเภท โดยคลาสนี้ทำหน้าที่เก็บข้อมูลพื้นฐานที่ใช้ร่วมกัน ได้แก่ username, password, name, email และ type

```
import java.io.Serializable;

public class User implements Serializable {
    private static final long serialVersionUID = 1L;

    protected String username;
    protected String password;
    protected String name;
    protected String email;
    protected String type;

    public User(String username, String password, String name, String email) {
        this.username = username;
        this.password = password;
        this.name = name;
        this.email = email;
    }

    public User(String username, String password, String name) {
        this.username = username;
        this.password = password;
        this.name = name;
        this.email = "";
    }

    public User() {
    }

    public boolean login(String username, String password) {
        return this.username.equals(username) && this.password.equals(password);
    }

    public void logout() {
    }
}
```

2. คลาส Driver (Subclass)

คลาส Driver ซึ่งอยู่ในไฟล์ src/model/Driver.java เป็นคลาสลูกที่สืบทอดจากคลาส User โดยใช้คำสั่ง extends User ทำให้สามารถใช้งานข้อมูลและเมธอดจากคลาส User ได้ทันที และสามารถเพิ่มข้อมูลเฉพาะของคนขับได้

```
import java.io.Serializable;

public class Driver extends User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String busId = "-";
    private Schedule currentSchedule;

    // --- Constructor ---
    public Driver(String username, String password, String name, String email) {
        super(username, password, name, email);
        this.setType("Driver");
    }

    public Driver(String name, String email, String password) {
        super(email, password, name, email);
        this.setType("Driver");
    }
}
```

3. การใช้งาน Inheritance ในการตรวจสอบก่อนเข้าเมนู

ระบบจะสร้างอ็อบเจกต์จากคลาสลูกที่สืบทอดจาก User และเรียกใช้เมธอด login() จากคลาส User เพื่อตรวจสอบข้อมูลก่อนเข้าใช้งานเมนู

Driver :

```
import java.io.Serializable;

public class Driver extends User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String busId = "-";
    private Schedule currentSchedule;
```

Passenger :

```
package model;

import java.io.Serializable;

public class Passenger extends User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String studentID;
    private List<Booking> myBookings;

    public Passenger(String name, String email, String password, String studentID) {
        super(name, email, password);
        this.studentID = studentID;
        this.myBookings = new ArrayList<>();
    }
}
```

การใช้แนวคิด Polymorphism ในการพัฒนา

ระบบถูกออกแบบโดยใช้แนวคิด Polymorphism ซึ่งเมธอดหลักชื่อ login() ถูกกำหนดไว้ในคลาสแม่ (User) เพื่อรับค่า username และ password แต่ถูกนำมาเขียนทับ (Override) ในคลาส Passenger เพื่อเปลี่ยนพฤติกรรมการทำงานให้เหมาะสมกับผู้ใช้งานประเภทนักศึกษา โดยเปลี่ยนจากการตรวจสอบ username ปกติ เป็นการตรวจสอบ studentID แทน ทำให้เมธอดชื่อเดียวกันสามารถทำงานแตกต่างกันได้ตามบริบทของวัตถุที่เรียกใช้

จากไฟล์: User

```
public class User implements Serializable {
    private static final long serialVersionUID = 1L;

    protected String username;
    protected String password;
    protected String name;
    protected String email;
    protected String type;

    // --- Constructor สำหรับรองรับการสร้าง Object หลากหลายรูปแบบ ---
    public User(String username, String password, String name, String email) {
        this.username = username;
        this.password = password;
        this.name = name;
        this.email = email;
    }

    public User(String username, String password, String name) {
        this.username = username;
        this.password = password;
        this.name = name;
        this.email = "";
    }

    public User() {}

    // --- จุดสำคัญของ Polymorphism ---
    // เมธอด login นี้ถูกออกแบบมาเพื่อให้คลาสลูก (เช่น Passenger)
    // สามารถนำไปเขียนทับ (Override) เพื่อเปลี่ยน Logic การตรวจสอบได้
    public boolean login(String username, String password) {
        return this.username.equals(username) && this.password.equals(password);
    }
}
```

จากไฟล์: Passenger

```
public class Passenger extends User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String studentID;
    private List<Booking> myBookings;

    // Constructor: (ชื่อ, อีเมล, รหัส, รหัสนักศึกษา)
    public Passenger(String name, String email, String password, String studentID) {
        // ▶ สอดตาม User: (Name, Email, Password)
        super(name, email, password);

        this.studentID = studentID;
        this.myBookings = new ArrayList<>();
    }

    @Override
    public boolean login(String inputID, String password) {
        return this.studentID.equals(inputID) && getPassword().equals(password);
    }

    // --- Booking Logic ---
    public Booking bookSeat(Schedule schedule, String seatID) {
        if (!schedule.getVehicle().isSeatAvailable(seatID)) return null;
        schedule.getVehicle().reserveSeat(seatID);
        String bookingID = "BK-" + System.currentTimeMillis();
        Stop start = (!schedule.getRoute().getStops().isEmpty()) ? schedule.getRoute().getStops().get(0) : null;
        Stop end = (!schedule.getRoute().getStops().isEmpty()) ? schedule.getRoute().getStops().get(schedule.getRoute().getStops().size() - 1) : null;
        Booking newBooking = new Booking(bookingID, this, schedule, seatID, start, end);
        this.myBookings.add(newBooking);
        return newBooking;
    }

    public List<Booking> viewHistory() { return myBookings; }
    public void cancelMyBooking(Booking booking) {
        if(booking != null) {
            booking.getSchedule().getVehicle().freeSeat(booking.getSeatNumber());
            booking.cancelTicket();
        }
    }

    // Getters / Setters
    public String getStudentID() { return studentID; }
    public void setId(String newId) { this.studentID = newId; }
    public String getId() { return this.studentID; }
    public void logout() {}
}
```

การใช้แนวคิด Composition ในการพัฒนา

ระบบถูกออกแบบโดยใช้แนวคิด Composition เพื่อสร้างวัตถุที่มีความซับซ้อน โดยการประกอบรวมวัตถุหลายส่วนเข้าด้วยกัน เห็นได้ชัดเจนในคลาส Schedule (ตารางเดินรถ) ที่ไม่ได้เก็บแค่ข้อมูลเวลา แต่ประกอบไปด้วยวัตถุ Route เพื่อระบุเส้นทาง, วัตถุ Vehicle เพื่อระบุรถที่ใช้, และวัตถุ Driver เพื่อระบุคนขับ ทำให้การจัดการรอบรถแต่ละรอบมีความยืดหยุ่นและสมบูรณ์ในตัวเอง

จากไฟล์: Schedule

```
public class Schedule implements Serializable {
    private static final long serialVersionUID = 1L;

    private String scheduleID;
    private LocalDateTime departureTime;
    private String status;

    private Route route;
    private Vehicle vehicle;
    private Driver driver;
    private List<Booking> bookingsOnThisSchedule;

    public Schedule(String scheduleID, Route route, Vehicle vehicle, LocalDateTime departureTime) {
        this.scheduleID = scheduleID;
        this.route = route;
        this.vehicle = vehicle;
        this.departureTime = departureTime;
        this.status = "ยังไม่เริ่ม";
        this.bookingsOnThisSchedule = new ArrayList<>();
    }
}
```

การใช้แนวคิด Aggregation ในการพัฒนา

แนวคิด Aggregation (การรวมกลุ่ม) คือความสัมพันธ์แบบ "Has-A" (คลาสหนึ่งมีอีกคลาสหนึ่งเป็นส่วนประกอบ) โดยที่วัตถุที่เป็นส่วนประกอบสามารถแยกออกจากกันได้ (Independent) และมีวงจรชีวิตที่แยกจากกัน ในโค้ดและเอกสารที่คุณส่งมา มีจุดที่แสดงถึงแนวคิดนี้ดังนี้ครับ

1. ในคลาส Booking (การจอง) คลาสนี้เป็นตัวอย่างของ Pure Aggregation เพราะเป็นการนำวัตถุที่สร้างไว้แล้วจากที่อื่น (เช่น ผู้โดยสาร, ตารางเดินรถ, จุดจอด) มาอ้างอิงรวมกันเพื่อสร้างข้อมูลการจอง หากรายการจองถูกลบไป ข้อมูลผู้โดยสารหรือตารางรถก็ยังคงอยู่ในระบบไม่ได้หายไปตามกัน

จากไฟล์: Booking

```
ic class Booking implements Serializable {
private static final long serialVersionUID = 1L;

// การใช้ Aggregation อ้างอิงไปยัง Object อื่นๆ
private Passenger passenger; // เชื่อมโยงกับผู้โดยสาร [cite: 345]
private Schedule schedule; // เชื่อมโยงกับตารางเดินรถ [cite: 346]
private Stop startStop, endStop; // เชื่อมโยงกับจุดจอดต้นทางและปลายทาง [cite: 348, 349]

// Constructor ที่รับ Object เข้ามาประกอบกัน
public Booking(String id, Passenger p, Schedule s, String seat, Stop start, Stop end) {
    this.passenger = p; [cite: 353]
    this.schedule = s; [cite: 354]
    this.seatNumbers = seat; [cite: 355]
    this.startStop = start; [cite: 356]
    this.endStop = end; [cite: 357]
    // ...
}
```


2. ในคลาส Booking (การจอง) คลาสนี้เป็นตัวอย่างของ Pure Aggregation เพราะเป็นการนำวัตถุที่สร้างไว้แล้วจากที่อื่น (เช่น ผู้โดยสาร, ตารางเดินรถ, จุดจอด) มาอ้างอิงรวมกันเพื่อสร้างข้อมูลการจอง หากรายการจองถูกลบไป ข้อมูลผู้โดยสารหรือตารางรถก็ยังคงอยู่ในระบบไม่ได้หายไปตามกัน

```
public class Passenger extends User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String studentID; [cite: 101]

    // Aggregation: ผู้โดยสาร "มี" รายการการจอง [cite: 102]
    private List<Booking> myBookings; [cite: 61, 377]

    public Passenger(String name, String email, String password, String studentID)
        super(name, email, password); [cite: 106]
        this.studentID = studentID; [cite: 107]
        this.myBookings = new ArrayList<>(); // เริ่มต้นรายการว่าง [cite: 109, 383]
    }
}
```

การใช้แนวคิด Encapsulation ในการพัฒนา

ระบบถูกออกแบบโดยใช้แนวคิด Encapsulation เพื่อปกป้องความถูกต้องของข้อมูล โดยการกำหนดให้ตัวแปรสำคัญเป็น private เช่น ในคลาส Vehicle ตัวแปร seatMap (ผังที่นั่ง) ถูกซ่อนไว้ไม่ให้เข้าถึงโดยตรงจากภายนอก หากต้องการจองที่นั่งจะต้องทำผ่านเมธอด reserveSeat() เท่านั้น ซึ่งภายในเมธอดจะมีการตรวจสอบเงื่อนไข isSeatAvailable() ก่อนเสมอ เพื่อป้องกันข้อมูลที่ผิดพลาดหรือการจองซ้ำซ้อน

จากไฟล์: Vehicle

```
public class Vehicle implements Serializable {
    private static final long serialVersionUID = 1L;

    private String busID;
    private int capacity;
    private Map<String, Boolean> seatMap; // true=ว่าง, false=ไม่ว่าง

    public Vehicle(String busID, int capacity) {
        this.busID = busID;
        this.capacity = capacity;
        this.seatMap = new HashMap<>();

        // สร้างที่นั่ง A1-D4
        for (char row = 'A'; row <= 'D'; row++) {
            for (int i = 1; i <= 4; i++) {
                seatMap.put(row + "" + i, true);
            }
        }
    }
}
```

การใช้แนวคิด Data Structures ในการพัฒนา

การเลือกใช้ Collection Framework ที่เหมาะสมกับโจทย์ เช่น การใช้ HashMap ในการจัดการที่นั่ง แทนที่จะใช้ Array ธรรมดา เพื่อความเร็วในการค้นหาและตรวจสอบสถานะ

จากไฟล์ Vehicle

```
public class Vehicle implements Serializable {
    private static final long serialVersionUID = 1L;

    private String busID;
    private int capacity;
    private Map<String, Boolean> seatMap; // true=ว่าง, false=ไม่ว่าง

    public Vehicle(String busID, int capacity) {
        this.busID = busID;
        this.capacity = capacity;
        this.seatMap = new HashMap<>();

        // สร้างที่นั่ง A1-D4
        for (char row = 'A'; row <= 'D'; row++) {
            for (int i = 1; i <= 4; i++) {
                seatMap.put(row + "" + i, true);
            }
        }
    }

    public boolean isSeatAvailable(String seatID) {
        return seatMap.getOrDefault(seatID, false);
    }

    public void reserveSeat(String seatID) {
        if (isSeatAvailable(seatID)) {
            seatMap.put(seatID, false); // ลุคจอง
        }
    }

    public void freeSeat(String seatID) {
        seatMap.put(seatID, true); // คืนมาว่าง
    }

    public String getBusID() { return busID; }
    public int getCapacity() { return capacity; }
    public Map<String, Boolean> getSeatMap() { return seatMap; }
}
```

การใช้แนวคิด Design Patterns ในการพัฒนา

การนำรูปแบบการแก้ปัญหามาตรฐานมาใช้ โดยเฉพาะ Observer Pattern เพื่อจัดการการแจ้งเตือนสถานะแบบ Real-time

จากไฟล์ StationStatusManager

```
public static void setDepartedStation(int index) {
    markDeparted(index);
}

/** ของเดิม: markDeparted ยังเก็บไว้ให้เรียกได้เหมือนกัน */
public static void markDeparted(int index) {
    if (index < 0 || index >= stations.size()) return;

    StationStatus s = stations.get(index);
    if (!s.isDeparted()) {
        s.setDeparted(true);
        // แจ้งทุกหน้าจอที่สมัครฟังว่า status เปลี่ยนแล้ว
        pcs.firePropertyChange("stations", null, null);
    }
}

/** เช็คสถานี่ index นี้ ออกแล้วหรือยัง */
public static boolean isDeparted(int index) {
    if (index < 0 || index >= stations.size()) return false;
    return stations.get(index).isDeparted();
}

/** รีเซ็ตทุกสถานีให้กลับมา "ยังไม่ออก" (ใช้ตอนเริ่มรอบถัดไป) */
public static void resetAll() {
    for (StationStatus s : stations) {
        s.setDeparted(false);
    }
    pcs.firePropertyChange("stations", null, null);
}

// -----
// จัดการ listener ให้หน้าจออื่นตามการเปลี่ยนแปลง
// -----

public static void addChangeListener(PropertyChangeListener l) {
    pcs.addPropertyChangeListener(l);
}

public static void removeChangeListener(PropertyChangeListener l) {
    pcs.removePropertyChangeListener(l);
}

public static int getCurrentBusIndex() {
    // TODO Auto-generated method stub
    return 0;
}
```

การนำเข้าข้อมูลจากไฟล์ (File Input / Database .dat)

ระบบมีการจัดเก็บข้อมูลแบบฐานข้อมูลไฟล์ โดยใช้ไฟล์ sut_transport_db_new.dat และนำข้อมูลกลับมาใช้งานเมื่อเริ่มโปรแกรม ผ่านการอ่าน Object ด้วย ObjectInputStream (เช่น รายการผู้โดยสาร คนขับ ตารางเวลา ฯลฯ) หากไฟล์ไม่มีอยู่หรืออ่านไม่สำเร็จ ระบบจะคืนค่า false

```
@SuppressWarnings("unchecked")
private boolean loadData() {
    File f = new File(DB_FILE);
    if (!f.exists()) return false;

    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(f))) {
        allPassengers = (List<Passenger>) in.readObject();
        allDrivers = (List<Driver>) in.readObject();
        allSchedules = (List<Schedule>) in.readObject();

        try {
            allBookings = (List<Booking>) in.readObject();
        } catch (Exception e) {
            System.out.println("Warning: Could not load bookings (New file version?)");
            allBookings = new ArrayList<>();
        }

        return true;
    } catch (Exception e) {
        System.out.println("Error loading data: " + e.getMessage());
        return false;
    }
}
```

การบันทึกข้อมูลลงไฟล์ (File Output / Database .dat)

เมื่อมีการเปลี่ยนแปลงข้อมูล เช่น สมัครสมาชิก จองตั๋ว หรือแก้ไขตารางเวลา ระบบจะบันทึกข้อมูลกลับลงไฟล์ .dat ด้วย ObjectOutputStream เพื่อเก็บสถานะล่าสุดของระบบไว้ใช้งานรอบถัดไป

```
public void saveData() {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(DB_FILE))) {
        out.writeObject(allPassengers);
        out.writeObject(allDrivers);
        out.writeObject(allSchedules);
        out.writeObject(allBookings);

        System.out.println("SystemManager: Data saved successfully.");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

การใช้แนวคิด Source ในการพัฒนา

1. แหล่งกำเนิดสถานะการเคลื่อนที่ (Movement Status Source) คือแนวคิดการสร้างแหล่งข้อมูลกลางเพื่อระบุสถานะ "สด" ของการเดินทาง (กำลังวิ่งหรือจอด) ซึ่งแยกออกจากข้อมูลตารางเวลาปกติ เพื่อใช้ในการแจ้งเตือนหน้าจอ UI แบบ Real-time

จากไฟล์ BusStatus

```
public class BusStatus {
    // แหล่งข้อมูลดัชนีสถานีปัจจุบัน
    public static int currentStationIndex = 0;

    // แหล่งระบุสถานะการวิ่ง (true = กำลังไปสถานีหน้า, false = จอด)
    public static boolean isEnRoute = false;

    // เมธอดที่เป็นต้นกำเนิดการเปลี่ยนสถานะเมื่อถึงสถานี
    public static void arriveAtNextStation() {
        currentStationIndex = (currentStationIndex + 1) % 7;
        isEnRoute = false;
        notifyAllScreens();
    }

    // เมธอดที่เป็นต้นกำเนิดการเปลี่ยนสถานะเมื่อรถออก
    public static void departFromStation() {
        isEnRoute = true;
        notifyAllScreens();
    }
}
```

2. แหล่งกำเนิดสถานะการเคลื่อนที่ (Movement Status Source) คือแนวคิดการสร้างแหล่งข้อมูลกลางเพื่อระบุสถานะ "สด" ของการเดินทาง (กำลังวิ่งหรือจอด) ซึ่งแยกออกจากข้อมูลตารางเวลาปกติ เพื่อใช้ในการแจ้งเตือนหน้าจอ UI แบบ Real-time

จากไฟล์ Stop

```
public class Stop implements Serializable {
    private String stopID; // แหล่งระบุรหัสสถานี
    private String stopName; // แหล่งระบุชื่อสถานี

    public Stop(String stopID, String stopName) {
        this.stopID = stopID;
        this.stopName = stopName;
    }

    public String getStopID() { return stopID; }
    public String getStopName() { return stopName; }
}
```

การใช้แนวคิด Search ในการพัฒนา

1. การค้นหาข้อมูลแบบเจาะจงด้วยคีย์ (Key-based Search)เป็นการใช้โครงสร้างข้อมูล Map เพื่อค้นหาข้อมูลที่นิ่งได้อย่างรวดเร็ว โดยอาศัยรหัสที่นิ่งเป็นคีย์ในการเข้าถึงข้อมูลทันทีจากไฟล์ Vehicle

```
public class Vehicle implements Serializable {
    private static final long serialVersionUID = 1L;

    private String busID;
    private int capacity;
    private Map<String, Boolean> seatMap; // true=ว่าง, false=ไม่ว่าง

    public Vehicle(String busID, int capacity) {
        this.busID = busID;
        this.capacity = capacity;
        this.seatMap = new HashMap<>();

        // สร้างที่นั่ง A1-D4
        for (char row = 'A'; row <= 'D'; row++) {
            for (int i = 1; i <= 4; i++) {
                seatMap.put(row + "" + i, true);
            }
        }
    }

    public boolean isSeatAvailable(String seatID) {
        return seatMap.getOrDefault(seatID, false);
    }

    public void reserveSeat(String seatID) {
        if (isSeatAvailable(seatID)) {
            seatMap.put(seatID, false); // ถูกจอง
        }
    }

    public void freeSeat(String seatID) {
        seatMap.put(seatID, true); // คืนมาว่าง
    }

    public String getBusID() { return busID; }
    public int getCapacity() { return capacity; }
    public Map<String, Boolean> getSeatMap() { return seatMap; }
}
```

2. การค้นหาเชิงเส้นและตรวจสอบสถานะ (Linear Search & Status Checking)

การค้นหาข้อมูลใน List เพื่อหาตำแหน่งปัจจุบันของรถบัส หรือค้นหาข้อมูลสถานที่ที่ตรงตามเงื่อนไขที่กำหนดจากไฟล์ StationStatusManager

```
public class StationStatusManager {
    private static List<StationStatus> stations;
    private static PropertyChangeSupport pcs;

    static {
        stations = new ArrayList<>();
        pcs = new PropertyChangeSupport(StationStatusManager.class);
        stations.add(new StationStatus("เทคโนโลยีธานี", "08:30 - 08:35"));
        stations.add(new StationStatus("The Mall", "09:00 - 09:10"));
        stations.add(new StationStatus("Terminal 21", "09:15 - 09:20"));
        stations.add(new StationStatus("บขส. ใหม่", "09:30 - 09:35"));
        stations.add(new StationStatus("Central", "10:00 - 10:15"));
        stations.add(new StationStatus("ลานย่าโม", "10:50 - 11:00"));
        stations.add(new StationStatus("กลับถึงเทคโนโลยีธานี", "11:40 - 11:50"));
    }

    public List<StationStatus> getStations() {
        return Collections.unmodifiableList(stations);
    }

    public int getCurrentBusIndex() {
        for(int i = 0; i < stations.size(); ++i) {
            if (!((StationStatus)stations.get(i)).isDeparted()) {
                return i;
            }
        }
        return -1;
    }

    public void resetAll() {
        Iterator<StationStatus> var1 = stations.iterator();
        while(var1.hasNext()) {
            StationStatus s = var1.next();
            s.setDeparted(false);
        }
        pcs.firePropertyChange("stations", null, stations);
    }
}
```


3. การค้นหาความสัมพันธ์ผ่านการอ้างอิงอ็อบเจกต์ (Object Reference Lookup)

เป็นการค้นหาข้อมูลที่เชื่อมโยงกันระหว่างหลายคลาส เช่น เมื่อทราบข้อมูลการจอง (Booking) จะสามารถสืบค้นย้อนกลับไปหาผู้โดยสารและตารางเดินรถได้

จากไฟล์ Booking

```
public class Booking implements Serializable {
    private static final long serialVersionUID = 1L;
    private String bookingID;
    private Passenger passenger;
    private Schedule schedule;
    private String seatNumber;
    private Stop startLocation;
    private Stop endLocation;

    public Booking(String bookingID, Passenger passenger, Schedule schedule, String seatNumber, Stop startLocation, Stop endLocation) {
        this.bookingID = bookingID;
        this.passenger = passenger;
        this.schedule = schedule;
        this.seatNumber = seatNumber;
        this.startLocation = startLocation;
        this.endLocation = endLocation;
    }

    public String getBookingID() {
        return this.bookingID;
    }

    public Passenger getPassenger() {
        return this.passenger;
    }

    public Schedule getSchedule() {
        return this.schedule;
    }

    public String getSeatNumber() {
        return this.seatNumber;
    }
}
```

4. การค้นหาข้อมูลเพื่อการยกเลิกและคืนทรัพยากร (Search for Resource Recovery)

การค้นหาข้อบกพร่องที่ต้องการในรายการ และสืบค้นข้อมูลต่อเนื่องเพื่อทำการอัปเดตสถานะในส่วนอื่นๆ ของระบบ

จากไฟล์ Passenger

```
public class Passenger extends User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String studentID;
    private List<Booking> myBookings;

    // Constructor: (ชื่อ, อีเมล, รหัส, รหัสนักศึกษา)
    public Passenger(String name, String email, String password, String studentID) {
        // ▶ สอดคล้อง User: (Name, Email, Password)
        super(name, email, password);

        this.studentID = studentID;
        this.myBookings = new ArrayList<>();
    }

    @Override
    public boolean login(String inputID, String password) {
        return this.studentID.equals(inputID) && getPassword().equals(password);
    }

    // --- Booking Logic ---
    public Booking bookSeat(Schedule schedule, String seatID) {
        if (!schedule.getVehicle().isSeatAvailable(seatID)) return null;
        schedule.getVehicle().reserveSeat(seatID);
        String bookingID = "BK-" + System.currentTimeMillis();
        Stop start = (!schedule.getRoute().getStops().isEmpty()) ? schedule.getRoute().getStops().get(0) : null;
        Stop end = (!schedule.getRoute().getStops().isEmpty()) ? schedule.getRoute().getStops().get(schedule.getRoute().getStops().size() - 1) : null;
        Booking newBooking = new Booking(bookingID, this, schedule, seatID, start, end);
        this.myBookings.add(newBooking);
        return newBooking;
    }

    public List<Booking> viewHistory() { return myBookings; }
    public void cancelMyBooking(Booking booking) {
        if(booking != null) {
            booking.getSchedule().getVehicle().freeSeat(booking.getSeatNumber());
            booking.cancelTicket();
        }
    }

    // Getters / Setters
    public String getStudentID() { return studentID; }
    public void setId(String newId) { this.studentID = newId; }
    public String getId() { return this.studentID; }
    public void logout() {}
}
```